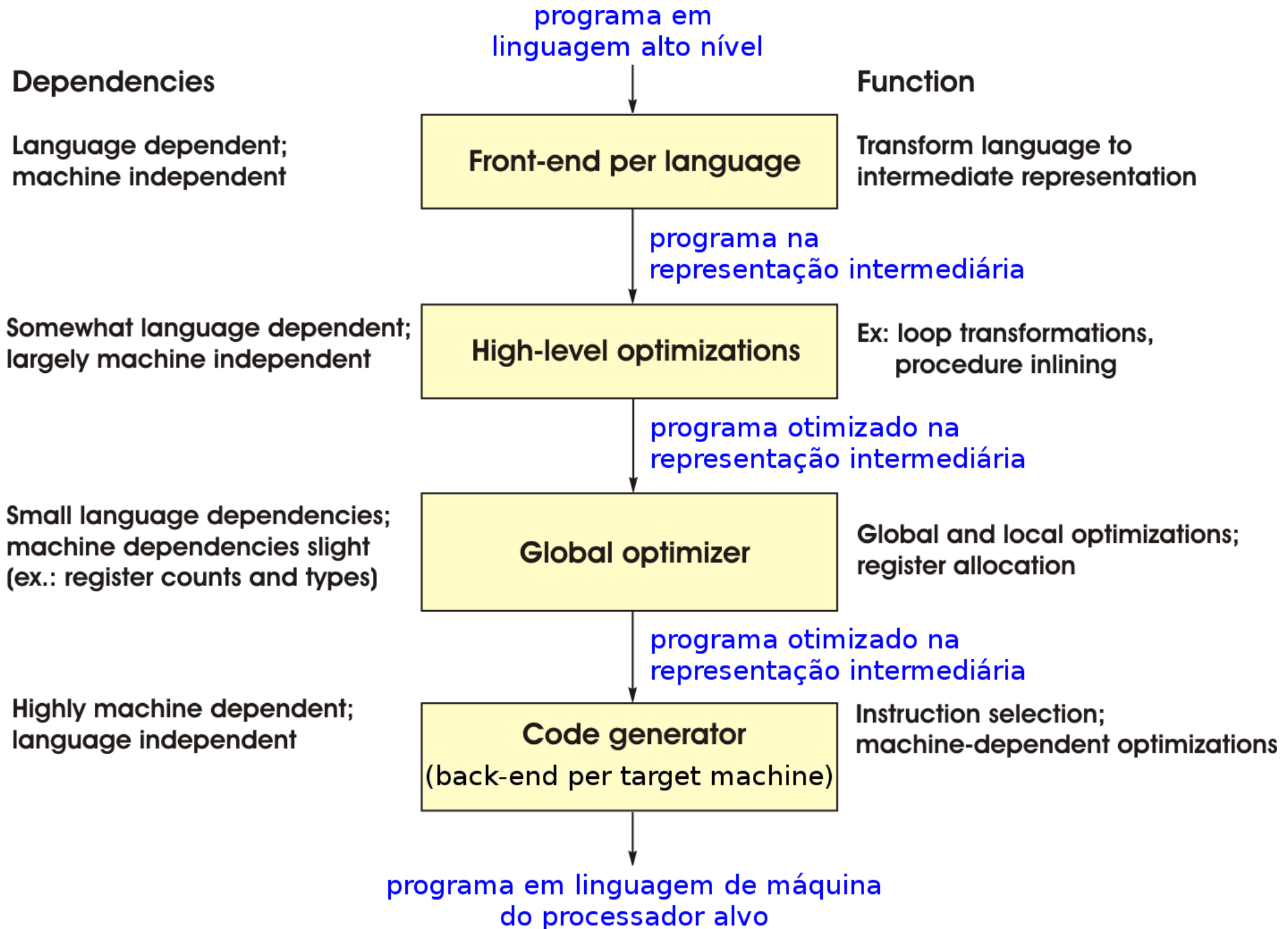


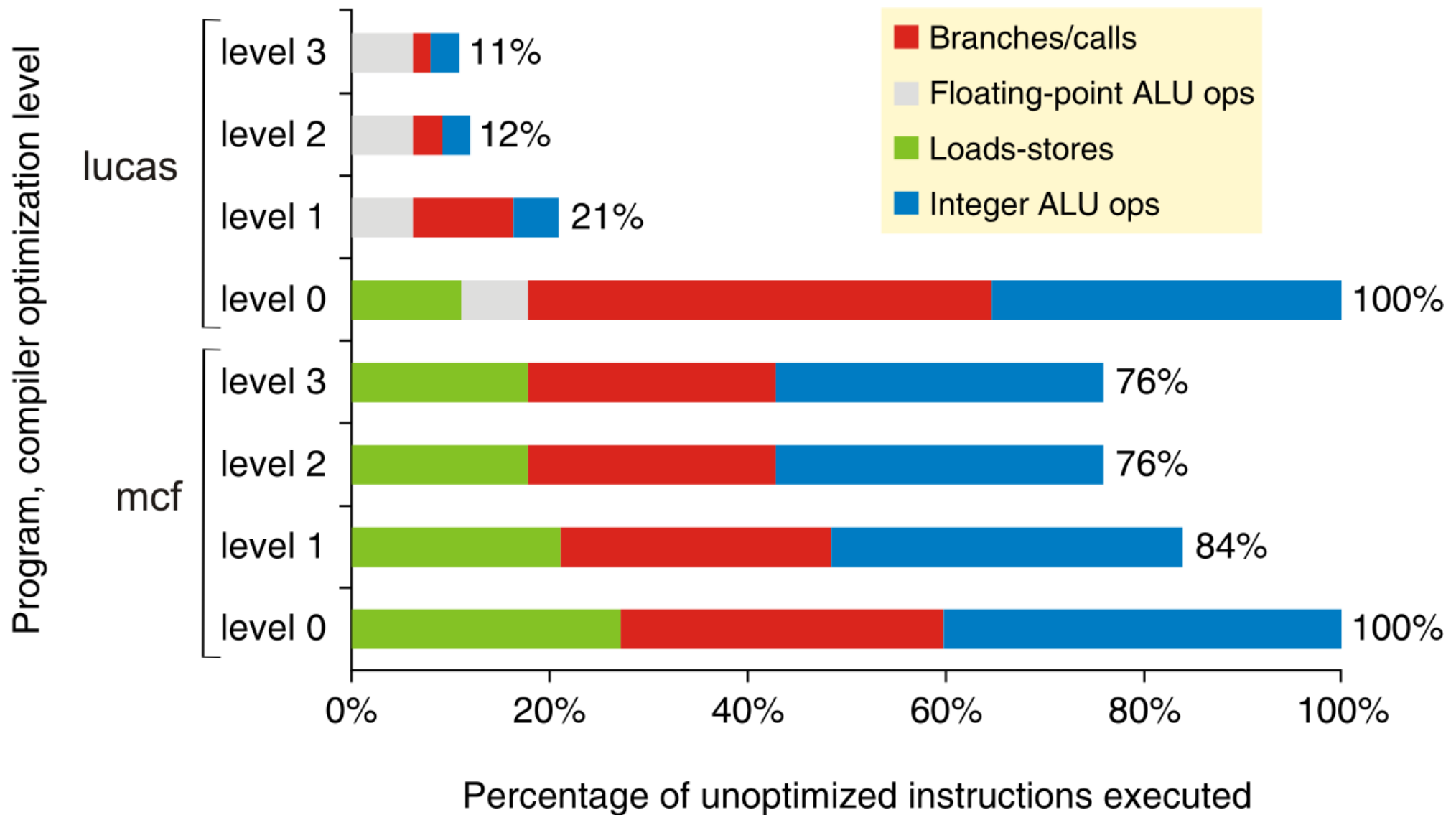
Técnicas Estáticas para Explorar ILP

- **Técnicas estáticas:**
 - Em geral, dependem do **compilador** para localizar ILP estaticamente, durante a compilação do programa
- **Ideia:** Usar tecnologia de compiladores para melhorar desempenho de:
 - Processadores com pipeline
 - Processadores com despacho múltiplo estático
- **Algumas técnicas:**
 - **Suporte em hardware para expor ILP na compilação**
 - Instruções condicionais
 - Instrução delayed branch
 - **Predição estática de desvios**
 - **Técnicas de compilação para expor ILP:**
 - Escalonamento estático de instruções
 - Loop unrolling
 - **Despacho múltiplo estático**

Estrutura dos Compiladores Atuais



Exemplo 1: Otimizações pelo Compilador



Suporte em Hardware para Expor ILP na Compilação

- **Instruções condicionais ou predicadas:**

- Instrução adicionada ao conjunto de instruções do processador
- Usada para eliminar desvio condicional (branch)
- Converte dependência de controle em dependência de dados:
 - **if-conversion**
- Instrução contém uma condição e uma operação

- **Execução:**

Se condição é verdadeira Então

Operação é realizada normalmente

Senão

Operação não é realizada (executa como um NOP)

Exemplo 2: Instrução *move condicional*

- Copia valor de registrador origem para destino, se condição é verdadeira:
 - `CMOVcond <reg destino>, <reg origem>, <reg condição>`
- **Instrução CMOVZ:**
 - Copia valor de registrador origem para destino, se registrador condição é 0
- Variáveis *a*, *b*, *c* alocadas nos registradores R1, R2, R3

- **Em C:**

```
if (a == 0)
    b = c ;
```

- **Com desvio condicional:**

```
        BNE    R1, R0, fim
        DADD   R2, R3, R0
fim:    ...
```

- **Com instrução condicional:**

```
CMOVZ   R2, R3, R1
... 
```

Instruções Condicionais

- **Vantagens:**

- Branch eliminado $\Rightarrow$ Não precisa de predição do branch
- Elimina atraso por conflito de controle
- Expõe mais ILP $\Rightarrow$ Pode melhorar desempenho

- **Limitações:**

- Útil para eliminar branch que “guarda” sequência de poucas instruções
- Se condição não é simples, instrução condicional pode gastar mais ciclos que instrução não condicional equivalente

Predição Estática de Desvios

- Predição **não** considera comportamento prévio da instrução de desvio (nesta execução do programa)
- Em geral, **realizada pelo compilador**, durante a compilação

- Para desvios condicionais

- **Exemplos:**

- **Predict not-taken:**

- Predição é sempre *not-taken*

- **Predict taken:**

- Predição é sempre *taken*
 - Útil apenas se endereço alvo calculado antes da condição de desvio

- **Predição baseada na direção do desvio:**

- Se desvio é para frente, predição é sempre *not-taken*
 - Se desvio é para trás, predição é sempre *taken*
 - Útil apenas se endereço alvo calculado antes da condição de desvio

```
    ...  
    BEQ R1, R2, fim  
fim:  ...  
    ...  
    ...  
loop: ...  
    ...  
    BEQ R1, R2, loop  
    ...
```

Escalonamento Estático de Instruções

- Compilador reordena instruções, sem modificar significado do programa
- **Ideias:**
 - Afastar instruções com dependências de dados $\Rightarrow$
Reduzir atrasos causados por dependências de dados
 - Utilizar instruções delayed branch e preencher delay slot $\Rightarrow$
Reduzir atrasos causados por dependências de controle
 - Encontrar instruções que possam ser iniciadas no mesmo ciclo do clock
(para processadores de despacho múltiplo estático)
- **Objetivo:**
 - Expor ILP $\Rightarrow$ Melhorar desempenho

Delayed Branch

- **Instrução de desvio condicional com atraso**

- Suporte do hardware que auxilia predição estática

- Delayed branch com **delay slot** = 1 $\Rightarrow$

- 1 instrução seguinte ao `branch` é executada, independente de desvio ser tomado ou não

...

BEQ R1, R2, fim

DADD ...

...

fim: DSUB ...

- **Compilador escalona instruções, preenchendo delay slot:**

- Compilador tenta preencher com **instrução “segura”** $\Rightarrow$ Nenhum atraso
 - Se não consegue, tenta preencher com **instrução especulada** (compilador tenta prever desvio tomado ou não tomado):
 - Se acerta $\Rightarrow$ Nenhum atraso
 - Se erra $\Rightarrow$ Atraso
 - Se não consegue, preenche com **NOP** $\Rightarrow$ Atraso

Exemplo 3: Escalonamento Estático de Instruções

```
for (i = 0 ; i < n ; i ++)  
    x[i] = x[i] + s ;
```

```
loop: LD      R3, 0    (R1)    # R3 = x[i]  
      DADD    R3, R3, R2      # R3 = R3 + R2  
      SD      R3, 0    (R1)    # x[i] = R3  
      DADDI   R1, R1, 4      # i ++  
      BNE     R1, R8, loop
```

- **Iterações independentes**
- **Cada iteração:**
 - 3 instruções para operar vetor
 - 2 instruções de **overhead do laço**: DADDI e BNE
- **Dentro da mesma iteração:**
 - **Dependências de dados verdadeiras** $\Rightarrow$ **Escalonamento limitado**
 - LD $\rightarrow$ DADD (R3)
 - DADD $\rightarrow$ SD (R3)
 - DADDI $\rightarrow$ BNE (R1)

Exemplo 3: Execução no Pipeline MIPS Básico

- **Sem escalonamento:** 8 ciclos por iteração $\Rightarrow$ 32 ciclos para 4 iterações
- **Com escalonamento e delayed branch:**
 - 5 ciclos por iteração $\Rightarrow$ 20 ciclos para 4 iterações

Sem escalonamento

Ciclo	Instrução executada
1	LD R3, 0 (R1)
2	atraso
3	DADD R3, R3, R2
4	SD R3, 0 (R1)
5	DADDI R1, R1, 4
6	atraso
7	BNE R1, R8, loop
8	atraso

Com escalonamento e delayed branch

Ciclo	Instrução executada
1	LD R3, 0 (R1)
2	DADDI R1, R1, 4
3	DADD R3, R3, R2
4	BNE R1, R8, loop
5	SD R3, -4 (R1)

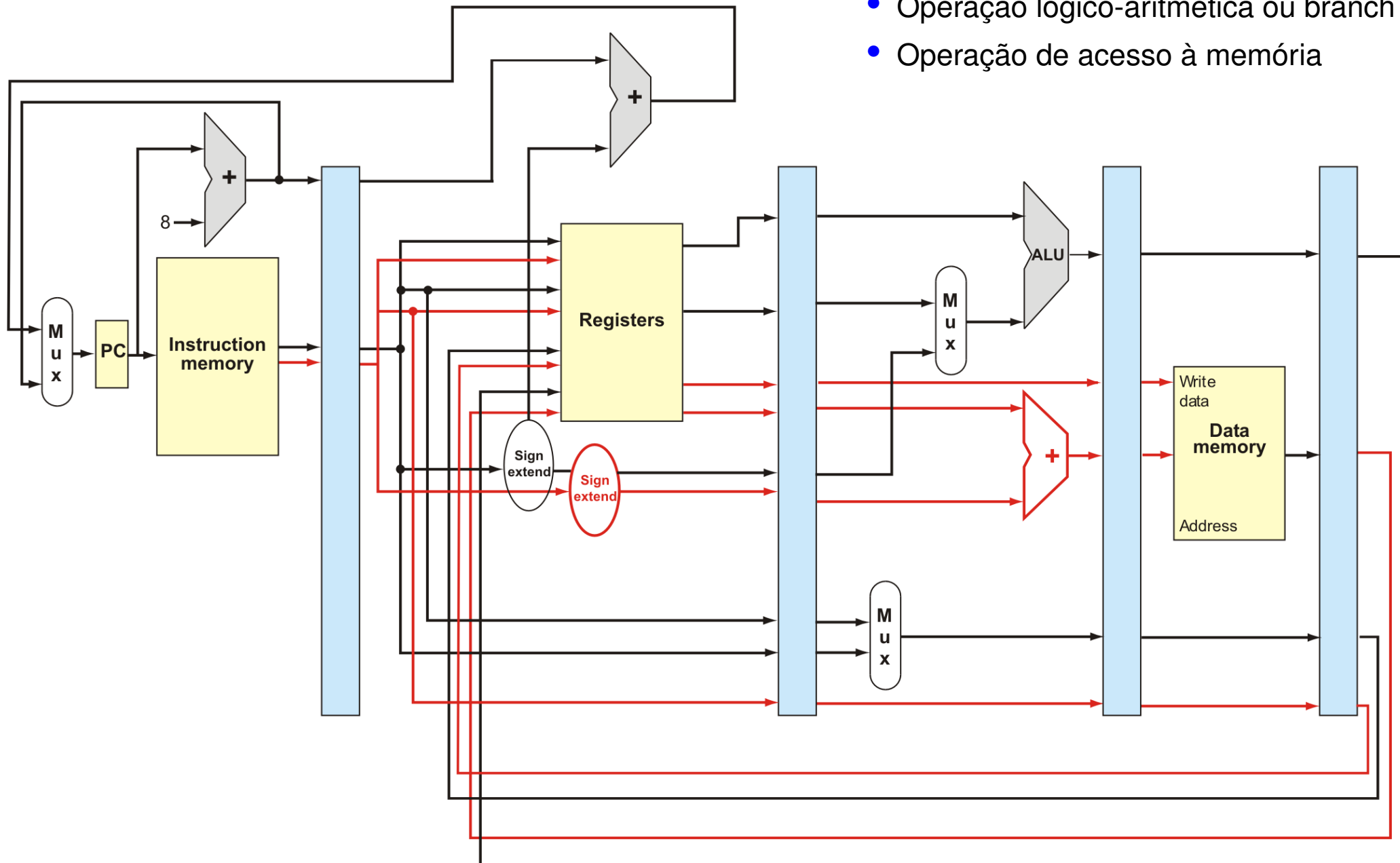
Processadores de Despacho Múltiplo (Multiple Issue)

- Despacham múltiplas instruções a cada ciclo do clock
- Possuem múltiplas unidades funcionais
- **Objetivo:**
 - Explorar mais ILP $\Rightarrow$ Obter $IPC > 1 \Rightarrow$ Melhorar desempenho
- 2 tipos:
 - Processador **Superscalar**: **Despacho dinâmico**, por **hardware**
 - Processador **VLIW** (Very Long Instruction Word) ou **EPIC** (Explicitly Parallel Instruction Computer)
 - **Despacho estático**: despacha n^o fixo de instruções por ciclo
 - **Compilador** encontra instruções para despachar simultaneamente
 - Escalonamento estático (pelo compilador)
 - Instrução formatada como:
 - Instrução longa com várias operações, ou
 - Pacote de instruções
 - Instrução longa tem campos para cada operação: **Operation slots**

Exemplo 4: Processador MIPS VLIW 2-issue

- **Instrução longa com 2 operações:**

- Operação lógico-aritmética ou branch
- Operação de acesso à memória



Exemplo 4: Instruções do Processador MIPS VLIW 2-issue

- **Instrução longa com 2 operações:**
 - Uma operação lógico-aritmética ou branch
 - Uma operação de acesso à memória

```
loop: LD      R3, 0    (R1)
      DADD    R3, R3, R2
      SD      R3, 0    (R1)
      DADDI   R1, R1, 4
      BNE     R1, R8, loop
```

- **Com escalonamento e delayed branch**

(e forwarding de DADD para SD):

- 4 ciclos por iteração $\Rightarrow$
16 ciclos para 4 iterações

- $IPC = \frac{5}{4} = 1,25$

- Ocupação dos *operation slots* = $\frac{5}{8} = \sim 62,5\%$

Ciclo

Instruções VLIW

	Operação load/store	Operação inteiros/branch
1	LD R3, 0 (R1)	DADDI R1, R1, 4
2	NOP	NOP
3	NOP	BNE R1, R8, loop
4	SD R3, -4 (R1)	DADD R3, R3, R2

Loop Unrolling

- **Técnica de otimização de código:**

- Realizada pelo **compilador**

- **Ideia:**

- Replicar corpo do laço várias vezes, ajustando controle do laço

- **Objetivos:**

- Reduzir overhead do laço
- Expor mais ILP $\Rightarrow$ Melhorar escalonamento das instruções
- Permitir que instruções de diferentes iterações sejam escalonadas juntas
- **Melhorar desempenho**

- **Passos:**

1. Replicação do corpo do laço (***unroll factor***)
2. Redução do **overhead do laço**
3. Renomeação de registradores

```
for (i=0 ; i<n ; i++)  
    ...
```

$\Downarrow$

```
for (i=0 ; i<n/4 ; i++)  
    ...  
    ...  
    ...  
    ...
```

Exemplo 5: Loop Unrolling

```
for (i = 0 ; i < n ; i ++)  
    x[i] = x[i] + s ;
```

```
loop: LD      R3, 0    (R1)    # R3 = x[i]  
      DADD   R3, R3, R2    # R3 = R3 + R2  
      SD     R3, 0    (R1)    # x[i] = R3  
      DADDI  R1, R1, 4      # i ++  
      BNE    R1, R8, loop
```

- **Iterações independentes**
- **Dentro da mesma iteração:**
 - **Dependências de dados verdadeiras:**
 - LD → DADD (R3)
 - DADD → SD (R3)
 - DADDI → BNE (R1)

Exemplo 5: Loop Unrolling – Passo 1

- Replicando corpo do laço: *unroll factor* = 4

```
loop:  LD      R3,    0 (R1)
      DADD    R3,    R3, R2
      SD      R3,    0 (R1)
      DADDI   R1,    R1, 4

      LD      R3,    0 (R1)
      DADD    R3,    R3, R2
      SD      R3,    0 (R1)
      DADDI   R1,    R1, 4

      LD      R3,    0 (R1)
      DADD    R3,    R3, R2
      SD      R3,    0 (R1)
      DADDI   R1,    R1, 4

      LD      R3,    0 (R1)
      DADD    R3,    R3, R2
      SD      R3,    0 (R1)
      DADDI   R1,    R1, 4

      BNE     R1,    R8, loop
```

- Entre iterações:
 - Dependências verdadeiras:
 - DADDI → LD (R1)
 - DADDI → SD (R1)
 - DADDI → DADDI (R1)
- Iterações devem executar em ordem
⇒ Escalonamento limitado

Exemplo 5: Loop Unrolling – Passo 2

- **Reduzindo overhead do laço:**

```
loop:  LD      R3,    0  (R1)
      DADD    R3,    R3, R2
      SD      R3,    0  (R1)

      LD      R3,    4  (R1)
      DADD    R3,    R3, R2
      SD      R3,    4  (R1)

      LD      R3,    8  (R1)
      DADD    R3,    R3, R2
      SD      R3,    8  (R1)

      LD      R3,   12  (R1)
      DADD    R3,    R3, R2
      SD      R3,   12  (R1)

      DADDI   R1,    R1, 16
      BNE     R1,    R8, loop
```

- **Entre iterações:**

- **Anti-dependência:**

- DADD / LD
- SD / LD

- **Dependências de saída:**

- LD / LD
- DADD / DADD

- Iterações devem executar em ordem
⇒ **Escalonamento limitado**

Exemplo 5: Loop Unrolling – Passo 3

- Renomeando registradores:

```
loop:  LD      R3,    0 (R1)
       DADD    R3,    R3, R2
       SD      R3,    0 (R1)

       LD      R4,    4 (R1)
       DADD    R4,    R4, R2
       SD      R4,    4 (R1)

       LD      R5,    8 (R1)
       DADD    R5,    R5, R2
       SD      R5,    8 (R1)

       LD      R6,   12 (R1)
       DADD    R6,    R6, R2
       SD      R6,   12 (R1)

       DADDI   R1,    R1, 16
       BNE     R1,    R8, loop
```

- Iterações independentes $\Rightarrow$
Permite escalonamento melhor

Exemplo 5: Execução no Pipeline MIPS Básico

- Com escalonamento e delayed branch: 14 ciclos por “iteração quádrupla”

```

loop:  LD      R3,    0 (R1)
       DADD    R3,   R3, R2
       SD      R3,    0 (R1)

       LD      R4,    4 (R1)
       DADD    R4,   R4, R2
       SD      R4,    4 (R1)

       LD      R5,    8 (R1)
       DADD    R5,   R5, R2
       SD      R5,    8 (R1)

       LD      R6,   12 (R1)
       DADD    R6,   R6, R2
       SD      R6,   12 (R1)

       DADDI   R1,   R1, 16
       BNE     R1,   R8, loop
    
```

Com escalonamento e delayed branch

Ciclo	Instrução executada
1	LD R3, 0 (R1)
2	LD R4, 4 (R1)
3	LD R5, 8 (R1)
4	LD R6, 12 (R1)
5	DADD R3, R3, R2
6	SD R3, 0 (R1)
7	DADD R4, R4, R2
8	SD R4, 4 (R1)
9	DADD R5, R5, R2
10	SD R5, 8 (R1)
11	DADDI R1, R1, 16
12	DADD R6, R6, R2
13	BNE R1, R8, loop
14	SD R6, -4 (R1)

Exemplo 5: Instruções do Processador VLIW 2-issue

- Com escalonamento e delayed branch:
 - 8 ciclos por “iteração quádrupla”
 - $IPC = \frac{14}{8} = 1,75$
 - Ocupação dos *operation slots* = $\frac{14}{16} = \sim 87,5\%$

Ciclo

Instruções VLIW

	Operação load/store	Operação inteiros/branch
1	LD R3, 0 (R1)	DADDI R1, R1, 16
2	LD R4, -12 (R1)	NOP
3	LD R5, -8 (R1)	DADD R3, R3, R2
4	LD R6, -4 (R1)	DADD R4, R4, R2
5	SD R3, -16 (R1)	DADD R5, R5, R2
6	SD R4, -12 (R1)	DADD R6, R6, R2
7	SD R5, -8 (R1)	BNE R1, R8, loop
8	SD R6, -4 (R1)	NOP

Desafios das Arquiteturas VLIW

- **Vantagens:**
 - **Melhoria do desempenho**
 - **Pouca complexidade no hardware do processador**
- **Desvantagens:**
 - **Usa mais registradores**
 - **Aumento do tamanho do código:**
 - Causado por:
 - Loop unrolling
 - Baixa ocupação dos operation slots
 - **Baixa compatibilidade de código:**
 - Código gerado usando:
 - Conjunto de instruções
 - Organização do processador (quantidade, tipo e latência das FUs)
 - ⇒ Dificuldade de migração para implementação diferente (sucessora)